

Summary of "LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism"

Esha Munagala (munagala), Chimaobi Okite (cokite), Alvaro Vega Hidalgo (alvarovh)

Problem and Motivation

There has been a growing line of work focused on increasing the context length of large language models and improving their ability to reason over long sequences. This has led to the emergence of models with context windows extending into the millions of tokens. While longer contexts enhance a model's capacity for complex reasoning and multi-document understanding, they also make inference extremely expensive in both memory and computation. This is because:

- key-value (KV) cache used during inference **grows linearly with the sequence length n** . For example, for a single 1M-token input, the KV cache in the LWM model can occupy about 488GB of memory, far exceeding the capacity of a single GPU.
- The **self-attention mechanism scales quadratically, $O(n^2)$** with respect to sequence length n , causing computation cost and latency to increase sharply as the context grows.

To handle these challenges, existing serving systems employ **model parallelism, sequence parallelism, or hybrid combinations**. However, the degree of parallelism (DoP) in these systems is fixed before serving begins, which makes them static and inefficient in practice. Since requests vary widely in both input length and inference phase (prefill versus decode), a fixed configuration cannot adapt effectively. Also in this fixed setting, short inputs waste GPU capacity when too many devices are allocated, while very long inputs can exceed memory limits or lead to high latency when too few GPUs are used.

As a result, current serving frameworks struggle to balance GPU utilization, latency, and throughput when processing variable-length requests, which makes deploying long-context models in real-world settings difficult and inefficient. To address this limitation, the paper asks a core question: How can we serve long-context LLMs efficiently on limited GPU resources, despite large and variable memory and computation demands, without fixing the parallelism statically? Their proposed answer is **LoongServe**, a system that introduces **Elastic Sequence Parallelism (ESP)** to dynamically adjust the degree of parallelism during inference. This in turn

allows the system to adapt to input length, phase, and resource availability, enabling elastic and resource-aware serving for long-context large language models.

Related Works

Existing research on large language model serving has focused mainly on improving inference efficiency through **memory management** and **parallel computation**, but most systems remain static in their configuration. Frameworks such as vLLM, Text Generation Inference (TGI), and DeepSpeed-Inference optimize batch scheduling, KV cache management, and throughput, yet they fix the number of GPUs per model at deployment. This static design cannot adapt to variable request lengths or changing workloads. In parallel, distributed inference techniques like **model parallelism**, **pipeline parallelism**, and **sequence parallelism** divide computation across devices to fit large models or long inputs, but the degree of parallelism (DoP) is also fixed once the model is initialized. These methods cannot dynamically reconfigure resources based on input length or inference phase.

Solution Overview

The paper proposes **LoongServe**, a serving system designed to efficiently handle long-context large language models by introducing **Elastic Sequence Parallelism (ESP)**. The key idea is that the number of GPUs assigned to a request, known as the degree of parallelism (DoP), should not remain fixed but instead be adjusted dynamically based on the input length, inference phase (prefill or decode), and available system resources.

LoongServe consists of two main components: an **elastic scheduler** that determines the optimal DoP for each request and an **elastic runtime** that redistributes computation and memory across GPUs when the DoP changes. During inference, LoongServe can increase the DoP for long or memory-intensive requests and reduce it for shorter ones, maintaining high GPU utilization while avoiding out-of-memory errors. It can even reconfigure the DoP between the prefill and decode phases, since these phases have very different resource requirements.

Given that workload variability across requests and inference stages requires adaptive resource allocation, by making sequence parallelism elastic rather than static, LoongServe achieves better memory efficiency, lower latency, and higher throughput without modifying the underlying model architecture. This adaptability allows it to efficiently serve long-context LLMs on limited GPU resources while maintaining scalability and stability.

LoongServe shows promising results over baselines; it achieves up to 4.64× higher total throughput and 4.00× higher input throughput compared to vLLM, up to 3.85× and 3.37× over DeepSpeed-MII and LightLLM, and up to 5.81× and 3.58× over DistServe, while maintaining lower latency and avoiding GPU memory limitations or interference between phases.

Limitations

1. LoongServe introduces additional runtime complexity due to the need to dynamically adjust the degree of parallelism (DoP) during inference. Reconfiguring GPU groups and redistributing KV caches requires careful synchronization and communication, which can introduce overhead and partially offset its efficiency gains.
2. LoongServe's effectiveness also depends on the accuracy of its cost model for predicting memory and compute usage across different input lengths and inference phases. The cost model is implemented as a lightweight analytical model trained on initial profiling data. This means that if workload patterns, model architectures, or GPU configurations change, the model would need to be re-profiled and retrained to maintain accuracy. Moreover, the linear formulation assumes near-linear relationships among

Future Research Directions

Future work could focus on developing a more adaptive cost model that learns online and captures non-linear resource interactions. Another direction is extending LoongServe to optimize DoP decisions across multiple concurrent requests and heterogeneous hardware. Combining LoongServe with model-level techniques such as sparse or linear attention could further improve efficiency for long-context LLM serving.

Summary of Class Discussion

- Looking at an evaluation graph, it looks like at a request rate of 1.5 it underperforms the baseline without ESP. Why?
 - Latency for the request is higher, so the SLO attainment at those higher request rates is lower.
- Why are they concerned about memory fragmentation in prefill-decode disaggregation?
 - There is no global memory attention, it's local per instance. A sequence of blocks has to fit in one single instance, sometimes this simply isn't possible and causes the so called memory fragmentation.
- They mention using an analytical model, was it trained online or offline?
 - It was trained offline.
- The authors claim that there's no communication overhead because they do the data migration during prefill?
 - You don't need to migrate any KV caches because of how they scale down, they just cache accordingly as instances move around without having to do any additional migration.
- The dispatching-batching order seems kind of arbitrary, can you do them at the same time to achieve better performance? It seems like it might be more optimal to do batching first

- The sequence seems like it can be optimized. This is the logic that the loongserve global manager uses, but the choice of the sequence is not really discussed and could likely be optimized.
- We talked previously about time per output token as a measure, this paper talks about latency so how does scaling affect time per output token?
 - They don't have any plots in the evaluation showing how this might affect time per output token or measures like that.
 - Follow up: how did they define SLO in the evaluation then?
 - They used the commonly used definition of median latency, didn't discuss it much.

Summary of "Cornserve: Efficiently Serving Any-to-Any Multimodal Models"

Esha Munagala (munagala), Chimaobi Okite (cokite), Alvaro Vega Hidalgo (alvarovh)

Problem and Motivation

Recently, there has been a surge in multimodal models: models that handle audio, video, and images in addition to text. These models can generate those different modes and/or understand them as input, and are referred to under the umbrella of Any-to-Any models.

However, an efficient way to online serve and deploy these models does not yet exist. This is because Any-to-Any models have two types of heterogeneity that introduce complexity when serving them. First, they have request type and computation path heterogeneity; this means that any given inference request can have different modalities for input and output, which leads to different components within the model being invoked, and a different computation path. Additionally, different components can be queried with different request rates, resulting in a lot of heterogeneity. Second, each of these different components has different GPU resource and performance characteristics, which means they have different ways to optimize throughput. Additionally, when all of the components are managed in a monolithic way, this can result in throughput overall being bottlenecked by the slowest component.

Because of these complexities, the existing ways that Any-to-Any models are served are stopgaps. Existing systems have limited support; they support only one type, such as vLLM supporting text-only output and xDiT focusing on image/video output from text input. They also choose a specific deployment strategy, which can be suboptimal because they may be designed only for text-only LLMs, or do not consider model and workload characteristics of individual components. Some recent works do describe disaggregation strategies to separate different computation phases or to separate different components, which are promising, but unilaterally applying them can still be harmful to throughput.

Because Any-to-Any models are so heterogenous in so many different ways, the best deployment strategy depends on the individual model and workload. Thus, the authors present Cornstarch, an online serving system for Any-to-Any models with a planner to determine the best deployment strategy, with the goal of maximizing throughput and resource efficiency, and overcoming the shortcomings of previous systems and methods.

Related Works

- **LLM serving systems and optimizations:** Related work focuses on either text-only LLMs or multimodal input-only LLMs. These works propose the disaggregation strategies

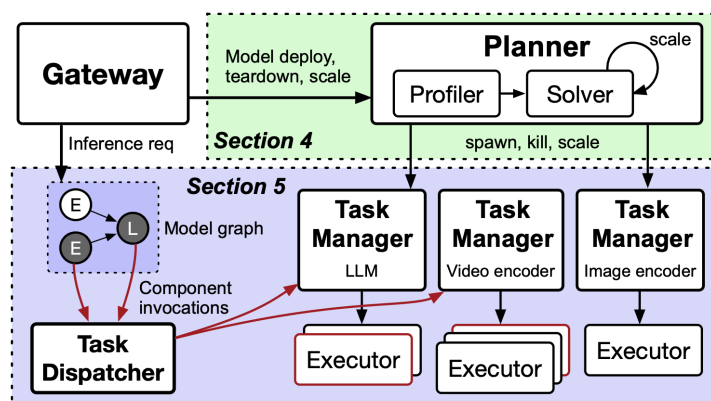
previously mentioned, like Prefill-Decode and Encode-Prefill-Decode. There are also efforts to scale multimodal LLMs through optimized containers. However, none of these related works address the inherent heterogeneity of Any-to-Any models.

- **Parallelizing large models for training:** Some early systems had manually devised strategies with expert input, like MegatronLM, DeepSpeed and G-Pipe. Later systems used automated planners like Alpa, GSPMD, Oobleck and WLB-LLM. Cornserve's planner selects the best out of a set of manually devised parallelism strategies.

Solution Overview

The authors propose a system called Cornserve, which will allow for efficient serving of Any-to-Any models and optimize GPU resource usage and maximize throughput. Cornserve's architecture has five components:

- A planner that allocates resources. This also includes a profiler to measure the throughput of each deployment option, to help the planner decide the best strategy.
- A gateway that serves as the entry point for control signals and inference strategies.
- A task dispatcher that sends model component invocations to executors.
- A task manager that is in charge of a set of task executors for a single model component deployment.
- A task executor that executes model components on a GPU.



The Cornserve system runs as such: a new model ready for deployment is sent to the gateway. Then, the gateway invokes the planner to figure out how to allocate resources; the planner in turn invokes the profiler to help it determine performance characteristics of each model component. After the planner completes planning, it creates task managers for each model component, and each task manager creates as

many task executors as the planner has dictated. During the model's runtime, based on either admin input or load monitoring, the planner can replan resource allocation and create or kill task managers and executors as needed. As the model is running and inference requests are sent, the gateway runs a mock of it with no actual computation performed to figure out the computation graph; it then sends that to the task dispatcher, which runs the request by dispatching it to components as needed.

One key insight that the Cornserve paper does is the cell abstraction; they consider a cell a group of GPUs that is of a power of 2 size. Using this abstraction when planning reduces the problem size that Cornserve has to consider dramatically, and also allows Cornserve to consider GPUs grouped in ways that they would be physically in real life, i.e. in the same rack or server.

Evaluating Cornserve, it shows up to 3.81x improvement in throughput and up to 5.79x improvement in latency compared to the state of the art. When running Cornserve on multimodal input models, it matches or outperforms the previously best deployment strategies. Cornserve also introduces request-static routing during its distributed runtime, which provides 3.7x higher throughput compared to dynamic load balancing.

Limitations

- The paper makes a simplification of GPU network topology for the planner by abstracting GPUs into power-of-2 numbers of cells. They assume that cells of reasonable sizes can be placed in a high affinity network domain like a server or a rack, but this is a limitation as models get large enough that they expand beyond a single server or rack and have to consider network topology.
- The record & replay method they describe assumes that control flow must be deterministic per request and assumes that the method called has no side effects because it gets called multiple times. Cornserve does not work if these assumptions are violated.
- Communication between data flows is not really considered, Cornserve assumes instant communication.

Future Research Directions

In the future, the limitations detailed above could be addressed. The cell-based approach could be expanded to consider network topology on larger-scaled models, or to evaluate how communication between dataflows affects the performance of Cornserve. Additionally, figuring out how to not require the assumptions that the record & replay method needs to work could expand the use cases for Cornserve.

Summary of Class Discussion

- What assumptions were made about the profiling workload, and if the runtime workload is different from the profiling workload what is the impact?
 - The solver would solve based on the input model graphs, it's easy to change the workflow by providing different input graphs
 - If there are significantly different workloads between offline planning and online serving, you have to rerun the profiler to reconfigure for a better plan
 - It assumes you have a good idea of what the workload will be when the model is in use.
- Did the authors explore other approximation algorithms for the NP-hard problem before deciding to use the cell based plan? At every stage there seems to be a lot of search-space pruning.
 - Today's datacenters put GPUs in a node or a rack and the cell approximation is used because it is a reflection of those cells of GPUs in a rack

- Allows Cornserve to consider network topology in the sense of GPUs bundled together when solving
 - Keeps the solution constrained into having a cell of GPUs that are close to each other and have less communication overhead bundled together
- Is it possible for optimal cell size to not be a power of 2? Are there tradeoffs from the constraint?
 - The power of 2 constraint is informed by the number of GPUs usually found physically in a rack, server etc
- The search space got reduced to $n + 1$, what are the performance impacts of restricting to that/can the user provide what they know?
 - If the user has known good combinations, Cornserve can use the combination. i.e. combining prefill and decode
- Online adaptation to changing workload: elaborate on that?
 - If the executor graph doesn't need changing, then that can be handled by the cells in the online phase. But if there's a LOT of change, you have to go back to the offline phase to replan.
 - We assume the workload distribution is the same, but the total amount of incoming traffic could be changed. It's more of a load change than a workload change. If distribution changes then you need to rerun the offline phase.
- Motivated work by saying multimodal models can have heterogeneous requests. On a high level, how would this approach work compared to existing systems that optimize for different modalities?
 - They were trying to scope it down by focusing on a specific category of model, even though as they continue to work on it it should become more general purpose. This was kind of done more to satisfy reviewers and make people more receptive to the argument.
 - It's a smaller step to take from what we have today, wanted to lean in on the trend of Any-to-Any models.
- Is Cornserve task abstraction generalizable enough for data-specific requests, that can change on a subsequent runthrough?
 - Task generalization works for requests that will stay the same every time it runs through the model
 - They have created another layer that doesn't rely on replay-record that wasn't talked about in the paper.